

Data Lakes & Data Integration - Projet Final

Cours EFREI 2025-2026

Yvann VINCENT
Machine Learning Engineer

Introduction

1 Objectifs du projet

Ce projet final vise à mettre en pratique l'ensemble des concepts que nous avons explorés durant ce cours de Data Lakes & Data Integration. Au-delà d'une simple validation des acquis, il représente une opportunité de créer un projet concret et potentiellement utile pour votre portfolio.

L'objectif est de concevoir et d'implémenter un data lake de A à Z, en partant de l'ingestion des données jusqu'à leur exposition via une API. Ce projet vous permettra de démontrer votre compréhension des architectures de données modernes et votre capacité à mettre en place des pipelines de données robustes et efficaces.

2 Exigences de base

2.1 Architecture du Data Lake

Le cœur de votre projet reposera sur un data lake structuré en plusieurs zones distinctes. Je vous laisse la liberté d'utiliser la structure raw/staging/curated à laquelle nous nous sommes cantonnés en cours, ou alors une structure différente que je vous laisserai le soin de justifier.

Je vous contrains simplement à utiliser elasticsearch ou un s3 pour la zone raw. Pour le reste, je ne vous impose rien. A vous de rechercher et choisir une technologie qui vous semble pertinente par rapport aux données que vous traitez.

Je vous laisserai également le soin de construire les pipelines de transformation entre les différentes zones.

Cette année vous avez eu vos premiers cours de Machine Learning et de Deep Learning. C'est une bonne opportunité de pratiquer et utiliser les choses que vous avez appris dans vos autres cours de façon transverse, par exemple intégrer un CNN pour de l'analyse d'image :)

2.2 Source de données

Je vous demande d'utiliser deux sources de données : un dataset fichier et une source issue d'une API. Je vous laisse une liberté totale sur le choix de votre source de données. Vous pouvez opter pour :

- Des datasets disponibles sur Hugging Face
- Des APIs publiques de votre choix
- Vos propres datasets d'images, de vidéos, ou tout autre type de données qui vous intéresse
- En bref, ce qui vous semble pertinent pour votre portfolio personnel

Cette flexibilité vous permet de travailler sur un sujet qui vous passionne ou qui pourrait être utile dans vos projets futurs. Par exemple, si vous êtes intéressé par le traitement d'images, vous pourriez construire un data lake pour automatiquement analyser toutes les photos qu'il ingère.

2.3 Idées de projets et Sources d'inspiration

Pour vous aider à vous projeter et à construire un projet solide qui fera la différence sur votre CV, voici une sélection d'idées de projets et de sources de données. L'intérêt d'un Data Lake est de savoir gérer des données hétérogènes (fichiers audio, images, séries temporelles, signaux physiologiques, texte brut) et de les valoriser à travers vos pipelines. N'hésitez pas à mobiliser vos compétences en Machine Learning et Deep Learning !

2.3.1 1. Traitement Audio & Analyse Musicale

- **Sources de données :**
 - **Zenodo / Free Music Archive (FMA)** : Des milliers de morceaux de musique libres de droits avec leurs métadonnées.
 - **Freesound API** : Une base de données collaborative de sons d'ambiance, d'effets sonores et de bruits du quotidien.
 - **Spotify API** : Idéal pour récupérer des métadonnées riches et des caractéristiques audio pré-calculées (*danceability, energy, tempo*).
 - **Soundcharts API** : Idéal pour récupérer des données sur vos artistes préférés toutes plateformes confondues. Vous voulez faire un moteur de forecast pour prédire et stocker le nombre de streams mensuels de Drake ? Vous êtes au bon endroit.
- **Ce que vous pouvez en faire (Pipeline) :**
 - **Raw** : Stockage des fichiers audio bruts (.mp3 ou .wav) sur S3.
 - **Staging** : Extraction automatique de caractéristiques (features) à l'aide de la librairie Python **librosa** (spectrogrammes, coefficients MFCC, chroma features, tempo).
 - **Curated** : Vecteurs de caractéristiques nettoyés et prêts pour l'analyse. Vous pouvez y intégrer un modèle de Deep Learning (par exemple, un CNN entraîné sur les spectrogrammes) pour classifier automatiquement le genre musical, détecter l'instrument ou faire de l'analyse d'émotions (*mood detection*).

2.3.2 2. Computer Vision, Vidéosurveillance & Sécurité

- **Sources de données :**
 - **Kaggle / Hugging Face** : Nombreux datasets de vidéosurveillance (détection de vols, d'accidents, de feux de forêt).
 - **Flux IP / RTSP publics** : Vous pouvez simuler ou consommer des flux de caméras de surveillance routière ou météo accessibles publiquement, ou simplement utiliser votre webcam.
- **Ce que vous pouvez en faire (Pipeline) :**
 - **Raw** : Ingestion des frames vidéo brutes ou de logs d'événements.
 - **Staging** : Prétraitement des images (redimensionnement, normalisation) et passage dans un modèle de détection d'objets (comme YOLO ou un SSD pré-entraîné). Extraction des boîtes englobantes (*bounding boxes*) et segmentation.

- **Curated** : Logs de surveillance hautement structurés (ex: `{"timestamp": "14:32:01", "camera_id": 1, "persons_detected": 3, "anomaly_score": 0.85}`). Vous pouvez ainsi faire du flagging d'anomalies en temps réel (ex: détection d'intrusion ou de comportement suspect).

2.3.3 3. Astronomie & Exploration Spatiale

- **Sources de données :**

- **NASA APIs (Kepler & TESS)** : Courbes de lumière des étoiles pour la recherche d'exoplanètes.
- **NASA APOD (Astronomy Picture of the Day) API** : Images quotidiennes de l'espace avec descriptions textuelles.
- **Exoplanet Archive API** : Données tabulaires extrêmement riches sur toutes les exoplanètes confirmées à ce jour.

- **Ce que vous pouvez en faire (Pipeline) :**

- **Raw** : Images du ciel profond (fichiers `.fits` ou `.png`) ou séries temporelles brutes de luminosité d'étoiles.
- **Staging** : Alignement des images, réduction du bruit de fond par computer vision, ou normalisation des courbes de transit (luminosité au cours du temps).
- **Curated** : Détection automatique de transits d'exoplanètes en utilisant un réseau de neurones (CNN 1D ou LSTM). Vous loggez le mouvement et les caractéristiques physiques des exoplanètes détectées dans votre base finale.

2.3.4 4. Séries Temporelles, Finance & IoT

- **Sources de données :**

- **Monash Time Series Forecasting Archive** : Le dépôt de référence contenant des dizaines de datasets de séries temporelles (météo, trafic routier, consommation d'énergie, transactions bancaires).
- **Yahoo Finance API (yfinance) ou Alpha Vantage** : Données financières en temps réel et historiques.
- **Open-Meteo API / OpenAQ API** : Données météo et de qualité de l'air mondiales en temps réel.

- **Ce que vous pouvez en faire (Pipeline) :**

- **Raw** : Ingestion en continu (via Airflow) des payloads JSON bruts issus des APIs.
- **Staging** : Alignement temporel, gestion des valeurs manquantes, calcul d'indicateurs avancés (volatilité glissante, moyennes mobiles exponentielles, indicateurs techniques financiers).
- **Curated** : Flagging d'anomalies (via un modèle d'Isolation Forest ou un Autoencoder) pour détecter des comportements anormaux (ex: pics de pollution soudains, krachs flash) et maintien d'un *backdata* de volatilité pour l'analyse prédictive.

2.3.5 5. NLP, Réseaux Sociaux & Analyse de Sentiment

- **Sources de données :**
 - **Reddit API (PRAW) / Mastodon API** : Ingestion de posts et commentaires sur des sujets spécifiques en temps réel.
 - **Hugging Face Datasets** : Accès à des milliers de datasets textuels (avis clients, articles de presse, tweets historiques).
- **Ce que vous pouvez en faire (Pipeline) :**
 - **Raw** : Flux de textes bruts et métadonnées utilisateurs au format JSON.
 - **Staging** : Nettoyage du texte (tokenisation, retrait des *stop-words*, lemmatisation) et calcul de scores de sentiment à l'aide d'un modèle BERT léger ou de la librairie TextBlob/VADER.
 - **Curated** : Agrégation des tendances et évolution du sentiment général au cours du temps. Idéal pour construire un dashboard de suivi d'e-réputation ou détecter des signaux faibles (buzz, crises de communication).

2.3.6 6. Neurosciences & Signaux Physiologiques (EEG/ECG)

- **Sources de données :**
 - **PhysioNet** : La référence mondiale absolue pour les signaux physiologiques (contient des bases de données massives d'EEG pour l'épilepsie, d'ECG pour les arythmies cardiaques, ou de polysomnographie pour le sommeil).
 - **OpenNeuro** : Une plateforme de partage de données de neuroimagerie et d'EEG de pointe gratuites et ouvertes.
- **Ce que vous pouvez en faire (Pipeline) :**
 - **Raw** : Ingestion de fichiers de signaux bruts multicanaux au format `.edf`, `.bdf` ou `.fif` stockés sur S3.
 - **Staging** : Prétraitement du signal à l'aide de la librairie MNE-Python ou SciPy (filtrage passe-bande pour isoler les ondes Alpha/Beta/Theta, suppression des artefacts oculaires par ICA/PCA, et segmentation en fenêtres temporelles).
 - **Curated** : Extraction de caractéristiques fréquentielles (densité spectrale de puissance - PSD) ou temporelles. Vous pouvez y brancher un classifieur (comme un Random Forest ou un LSTM) pour détecter automatiquement des crises d'épilepsie, classifier les stades du sommeil, ou décoder des intentions de mouvement (interfaces cerveau-machine).

2.3.7 7. Smart Cities, Mobilité & Transports

- **Sources de données :**
 - **APIs Open Data (Paris, Lyon, Île-de-France Mobilités)** : Flux temps réel de disponibilité des vélos (Vélib), positions des bus, métros et trafic routier.

- **Flux GTFS-RT (General Transit Feed Specification Real-Time)** : Le standard mondial pour les données de transport en commun en temps réel.
- **Ce que vous pouvez en faire (Pipeline)** :
 - **Raw** : Ingestion continue de flux JSON ou de Protocol Buffers (protobuf) représentant l'état du réseau à l'instant t .
 - **Staging** : Indexation spatiale des coordonnées géographiques à l'aide de grilles géospatiales comme Uber H3 ou Google S2. Calcul des temps de parcours réels et alignement avec les horaires théoriques.
 - **Curated** : Base de données optimisée pour l'analyse spatio-temporelle. Vous pouvez entraîner un modèle de prédiction (ex: Graph Neural Networks ou XG-Boost) pour anticiper la saturation d'une station de vélos ou estimer précisément le retard d'un bus.

2.3.8 8. Sport, Wearables & IoT (Quantified Self)

- **Sources de données** :
 - **Strava API / Garmin Connect API** : Récupération de vos propres données d'activités sportives (courses à pied, sorties vélo) ou de données communautaires publiques.
 - **Fitbit API / Apple Health Export** : Données de santé quotidiennes (nombre de pas, fréquence cardiaque au repos, variabilité de la fréquence cardiaque - HRV).
- **Ce que vous pouvez en faire (Pipeline)** :
 - **Raw** : Ingestion de fichiers de traces GPS (`.gpx`, `.fit`) ou de payloads JSON de santé.
 - **Staging** : Parsing des fichiers binaires `.fit`, nettoyage des pertes de signal GPS, et calcul de métriques avancées (charge d'entraînement, zones de puissance, dérive cardiaque).
 - **Curated** : Profil de performance et de récupération de l'athlète. Vous pouvez concevoir un algorithme de détection de surentraînement (anomaly detection sur la HRV) ou un modèle prédictif pour estimer le temps de course sur un prochain marathon.

2.3.9 9. Génomique & Santé Moléculaire

- **Sources de données** :
 - **NCBI APIs (National Center for Biotechnology Information)** : Accès aux gigantesques bases de données de séquences de nucléotides et de protéines.
 - **Ensembl API / UCSC Genome Browser** : Données d'annotations de génomes complets pour de nombreuses espèces.
- **Ce que vous pouvez en faire (Pipeline)** :
 - **Raw** : Fichiers de séquences brutes au format `.fasta` ou `.fastq`.

- **Staging** : Alignement de séquences, filtrage des lectures de mauvaise qualité, et détection de mutations (variant calling).
- **Curated** : Séquences annotées et nettoyées. Vous pouvez utiliser du NLP (car l'ADN est un langage !) avec des modèles de type Transformer (comme DNABERT) pour classifier des familles de gènes, prédire la structure 3D d'une protéine ou identifier des prédispositions à certaines pathologies.

2.4 Pipeline d'intégration

Implémentez une pipeline d'intégration continue utilisant au choix :

- DVC
- Apache Airflow

La pipeline doit assurer la reproductibilité des transformations et l'alimentation automatisée du data lake.

Si vous utilisez Airflow, pensez à utiliser la fonctionnalité de scheduling pour ingérer des données depuis l'API à intervalle régulier. Pour les curieux, essayez d'expérimenter un peu avec X-COM si votre DAG devient un peu complexe.

2.5 API Gateway

L'API doit fournir les endpoints suivants :

- `/raw` : Accès aux données brutes
- `/staging` : Accès aux données intermédiaires
- `/curated` : Accès aux données finales
- `/health` : Vérification de l'état des services
- `/stats` : Métriques sur le remplissage des buckets et bases de données

Si vous n'utilisez pas la structure raw/staging/curated, ajoutez un endpoint GET pour chaque zone pertinente de votre Lake. Je rappelle que le but de cet API gateway est d'avoir une interface simple qui permet de récupérer les données ingérées sans avoir à s'embêter avec

3 Niveau Avancé

Pour ceux qui souhaitent se démarquer et relever un défi supplémentaire, j'ai créé un niveau avancé. Je tiens à préciser d'emblée que ce niveau est totalement optionnel. Vous pouvez obtenir une excellente note allant de 16 à 20-20 sans vous y attaquer. Ce niveau a été conçu pour ceux qui veulent aller plus loin et explorer des concepts plus avancés, typiquement ceux qui sont certains de vouloir s'investir dans le data engineering ou qui veulent un challenge supplémentaire just for the sake of it.

Je rappelle qu'à la clé, il y a des t-shirt data lakes pour les 3 meilleurs projets qui auront intégré les fonctionnalités suivantes :

3.1 Endpoints Avancés

3.1.1 L'endpoint /ingest

L'une des contraintes principales du niveau avancé concerne l'implémentation d'un endpoint d'ingestion de données depuis l'API Gateway. Votre API gateway devra exposer un endpoint `/ingest` capable d'accepter des données pertinentes à votre data source au format JSON et de les propager à travers votre pipeline d'ingestion.

Par exemple, si vous travaillez avec des données textuelles, votre endpoint pourrait accepter une structure JSON comme :

```
{
  "data": {
    "texts": ["Premier texte à analyser", "Deuxième texte à traiter", "..."]
  }
}
```

Un aspect crucial de cet endpoint sera la mesure de ses performances. Vous devrez chronométrer et documenter le temps d'exécution de votre pipeline pour :

- Un batch contenant un seul élément
- Un batch de 100 éléments

Ces mesures serviront de base de comparaison pour le second endpoint optimisé.

3.1.2 L'endpoint /ingest_fast

Le véritable défi technique réside dans l'implémentation de l'endpoint `/ingest_fast`. Dans un environnement de production, la performance est cruciale. Un data lake lent peut rapidement devenir un goulot d'étranglement pour toute l'infrastructure.

Cet endpoint doit offrir une amélioration de temps d'exécution d'au moins 30% par rapport à `/ingest`. Pour atteindre cet objectif, vous devrez réfléchir à des stratégies d'optimisation comme :

- L'utilisation de Numba pour accélérer les calculs numériques
- La vectorisation des opérations avec NumPy
- La parallélisation intelligente des traitements et le multithreading
- L'utilisation d'asynchronicité dans votre pipeline d'ingestion
- L'optimisation des requêtes et des accès aux données
- La mise en cache stratégique

La comparaison des performances devra être documentée, avec des mesures précises pour les mêmes tailles de batch que `/ingest`.

4 Critères d'Évaluation

4.1 Évaluation du niveau standard

Les critères d'évaluation de base incluront :

- La qualité de l'implémentation des trois couches du data lake
- La robustesse des scripts de transformation : est-ce que vous pensez à couvrir tous les cas ? est-ce que vous pensez à gérer les exceptions et renvoyer des messages d'erreurs pertinents ?
- La fiabilité de la pipeline d'intégration
- La qualité et la complétude de l'API Gateway : est-ce que tous les endpoints demandés sont présents et fonctionnent ?
- La documentation du projet : pensez à commenter votre code !
- La clarté du code et son organisation

4.2 Évaluation du niveau avancé

Pour le niveau avancé, je regarderai également :

- Les performances comparées des endpoints `/ingest` et `/ingest_fast`
- La créativité dans les solutions d'optimisation
- La qualité de la documentation technique des optimisations
- Bonus : L'originalité du choix des données et du domaine d'application

5 Livrables Attendus

Pour valider votre projet, vous devrez fournir :

- Un dépôt GitHub contenant l'intégralité du code source
- Une documentation technique détaillée expliquant :
 - L'architecture de votre solution
 - Les choix techniques effectués
 - Les procédures d'installation et de build de votre projet
 - Les résultats des tests de performance (pour le niveau avancé)
- Soyez exhaustifs dans votre README. Pas besoin de faire du Shakespeare, j'ai simplement besoin de savoir comment builder votre solution et l'utiliser en contrepartie de la liberté technique que je vous laisse pour réaliser le projet.

Pour ceux qui le souhaitent, je serai là pour review votre solution et vous donner des conseils lors de la séance du TP7 et du TP8.

Je vous souhaite bon courage, et j'espère que ce projet sera utile à votre portfolio.